Engineering

How DigitalOcean’s Agentic Inference Cloud powered by NVIDIA GPUs Achieved 67% Lower Inference Costs for Workato

By Rithish Ramesh, Karnik Modi, and Piyush Srivastava

Updated: March 3, 2026 • 11 min read

← Back to blog home

Table of contents →

Workato's AI Research Lab is focused on helping customers extend their production automation with agentic AI capabilities, systems that can reason, act, and orchestrate work across the business. At Workato's scale, processing 1 trillion automated workloads, LLM inference efficiency is a hard requirement: every millisecond of latency and every wasted GPU cycle directly impacts cost, throughput, and reliability. To make agentic workloads production-ready, the team needed an inference stack built for production scale – delivering predictable performance and unit economics at scale, not just raw compute.

DigitalOcean partnered with Workato's AI Research Lab team to design and tune this deployment on its Agentic Inference Cloud, using NVIDIA Dynamo with vLLM on DigitalOcean Kubernetes Service (DOKS). To support 100K-token context lengths without degrading performance, NVIDIA H200 GPUs were selected for their 141GB HBM3e memory capacity.

The memory footprint of the workload was around 125 GB (comprising the model weights, key value cache, and activation buffer), so a single NVIDIA H200 GPU is able to fit the whole footprint. However, the team used 8-way tensor parallelism per node to maximize sustained throughput and latency stability under a concurrent load.

DigitalOcean tested across two different configurations for Workato, and afterwards, the results for NVIDIA Dynamo + vLLM on DOKS showed:

- **Best in class queries-per-second** across all tested configurations
- **67% higher throughput per GPU** with **79% lower end-to-end latency** and **77% time-to-first-token** compared to different configurations on identical hardware
- **33% lower hardware cost** using a NVIDIA H200 GPU vs. a NVIDIA A100 GPU for equivalent performance
- **67% lower model cost** while using half the GPUs

The key here was to introduce key/value (KV)-aware routing in order to reduce redundancies and capture maximum value across performance and cost for the inference stack.

How LLMs Process Requests and Why It Gets Expensive at Scale

Before getting into the architecture decisions, it's worth understanding the mechanics that drive inference cost and why this is a complex problem that Workato needed to solve. Every LLM inference request goes through two phases:

- **Prefill** is where the model processes the entire input prompt and builds up internal memory, called key/value (KV) states, for every token it has read. This phase is compute-heavy and scales quadratically (O(n2)) with input sequence length. For long-context workloads (e.g 10K-100K token prompts), prefill can consume the majority of total inference cost. The primary reason for this is that the model needs to compute self-attention scores for every token against every other token in the prompt. As an example, if the prompt is 1000 tokens, the model performs roughly 1000 × 1000 attention operations. If the prompt is 100,000 tokens (as the case with Workato's workload), those operations jump to 10 billion. 100K token prefills require many floating point operations per second (FLOPs) and it can take several seconds of 100% GPU utilization, resulting in lower throughput per GPU directly contributing to cost.
- **Decode** is where the model generates tokens one at a time, using those cached KV states to predict each next token. This phase is memory bandwidth bound; performance of the decode phase directly impacts token streaming latency.

There are real-world workloads that share common input prefixes where a significant, identical "block" of text is reused across multiple requests. In enterprise SaaS applications (like Workato's AI Research Lab), there is often a high degree of prefix sharing across inference requests. As the GPU does prefill operations, it builds in-memory context (KV cache) which is expensive to build specifically for long-prompt workloads.

Now, if subsequent queries are all routed to separate GPUs, every GPU has to re-build the KV cache, resulting in redundant FLOPs being consumed which could have instead been used to serve other queries.

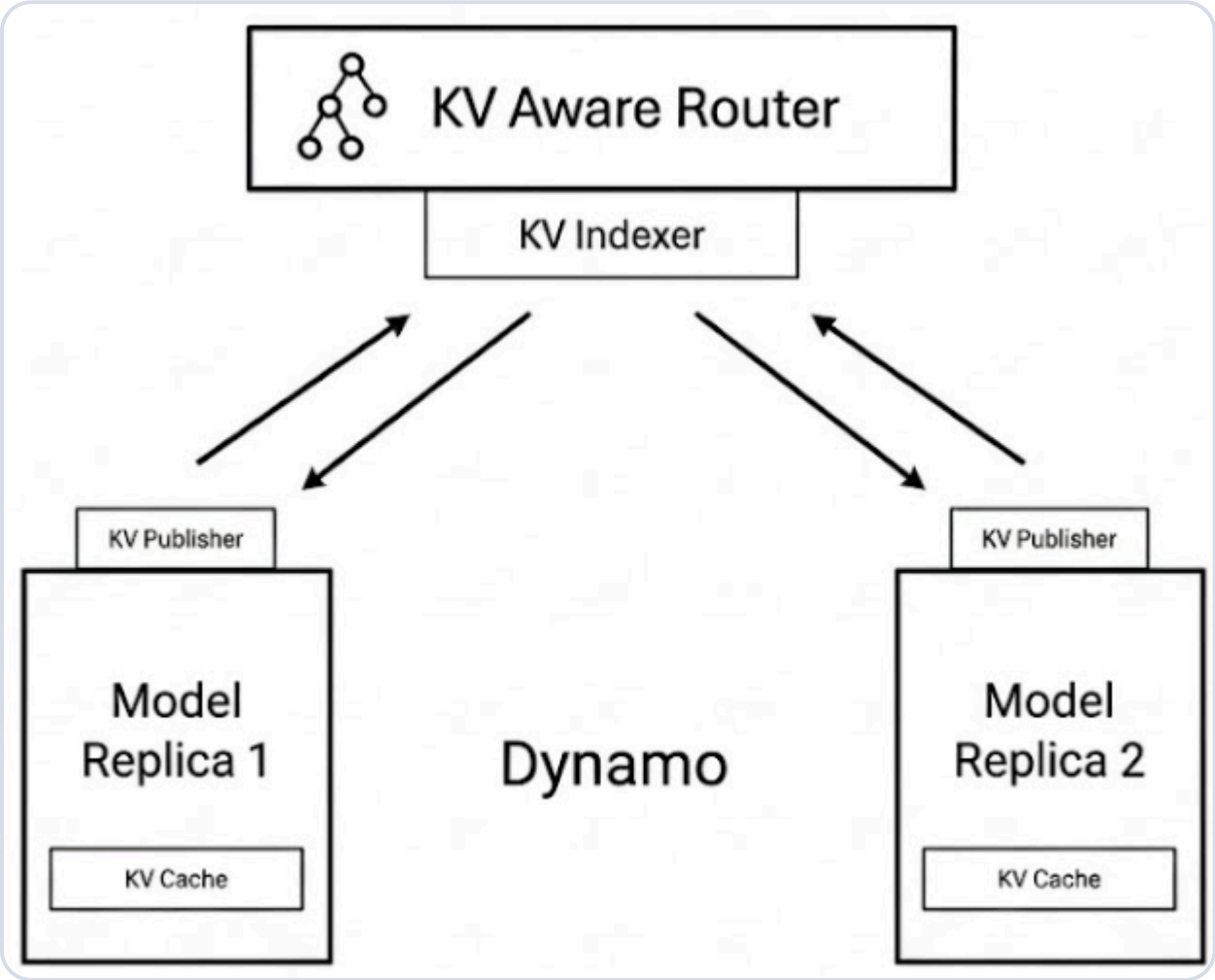
How KV-Aware Routing Addresses the Problem

KV-Aware routing is a technique which utilizes the commonality of prefixes and routes them to the same GPU. This helps by enabling the GPU to leverage a warm KV cache (often via RadixCache) to skip the compute-heavy prefill phase entirely.

This helps in dramatically reducing first token latency (TTFT) for the end user, and significantly increases the total throughput of the cluster by reclaiming GPU FLOPs which would have otherwise been spent on redundant prefill computations.

NVIDIA Dynamo with DOKS: The Orchestration Brain for KV-Aware Routing

NVIDIA Dynamo is an open-source, low-latency, modular inference framework designed to operate on top of individual inference engines. It is engine-agnostic and can orchestrate backends like vLLM (this is what we used here), TensorRT-LLM, and SGLang. Dynamo is not designed to make a single GPU faster. It is designed to prevent the cluster from doing redundant work and keep the right GPUs busy with the right phase of inference. In the context of Workato, we used Dynamo for its KV-aware routing capabilities.



NVIDIA Dynamo transforms standard LLM infrastructure by introducing a sophisticated orchestration layer that far exceeds the capabilities of a vanilla multi-node setup. At its core, Dynamo functions as a global scheduler with a comprehensive view of every GPU in the cluster, moving beyond the limitations of workers that only see their own local resources. This global perspective is managed by a cluster-level KV cache manager that

meticulously tracks which tokens reside on specific workers, identifies which blocks are hot or evictable, and determines the optimal time to reuse, offload, or recompute various cache segments.

The defining feature of this architecture is the KV-aware router, which replaces traditional, "blind" round-robin distribution with LLM-aware request routing. Rather than treating all workers as equal, the router utilizes a complex cost function to score candidate workers based on existing cache overlaps and the critical trade-offs between system metrics. Specifically, it balances the need to minimize prefill costs-improving Time to First Token (TTFT)-against the requirements for efficient decode performance, such as Inter-Token Latency (ITL) and Time Per Output Token (TPOT). By integrating this real-time awareness of global cache states, Dynamo ensures that routing decisions are globally optimal, enabling seamless KV cache offloading and maximum reuse across disparate requests.

However, deploying a cluster-aware inference stack like NVIDIA Dynamo introduces real operational complexity. You're no longer running a few vLLM servers behind a load balancer. You're running a distributed system with a routing frontend, instrumented worker backends, a KV cache manager, and real-time coordination between all of them. Getting that right requires both the right framework and platform powering it all. While NVIDIA Dynamo provides the logic, DOKS provides the execution environment that makes KV-aware routing possible. DOKS doesn't just 'run' the pods; its native service discovery and intelligent scheduling allow the Dynamo frontend to act as a cluster-wide dispatcher. This ensures that incoming requests are instantly routed to the specific GPU node where the relevant KV cache already lives,

This eliminates redundant computations and capture maximum value from the full inference stack, specifically:

- **Independent replica management** for the routing frontend and worker backends means the team could scale each layer separately as they tuned configurations.
- **Kubernetes-native service discovery** lets the router frontend find and route to worker pods cleanly, without custom networking work.
- **Operational simplicity**: DOKS abstracts away the control plane entirely, so Workato's engineers can focus on inference optimization, instead of Kubernetes orchestration.

Inference Stack Architecture

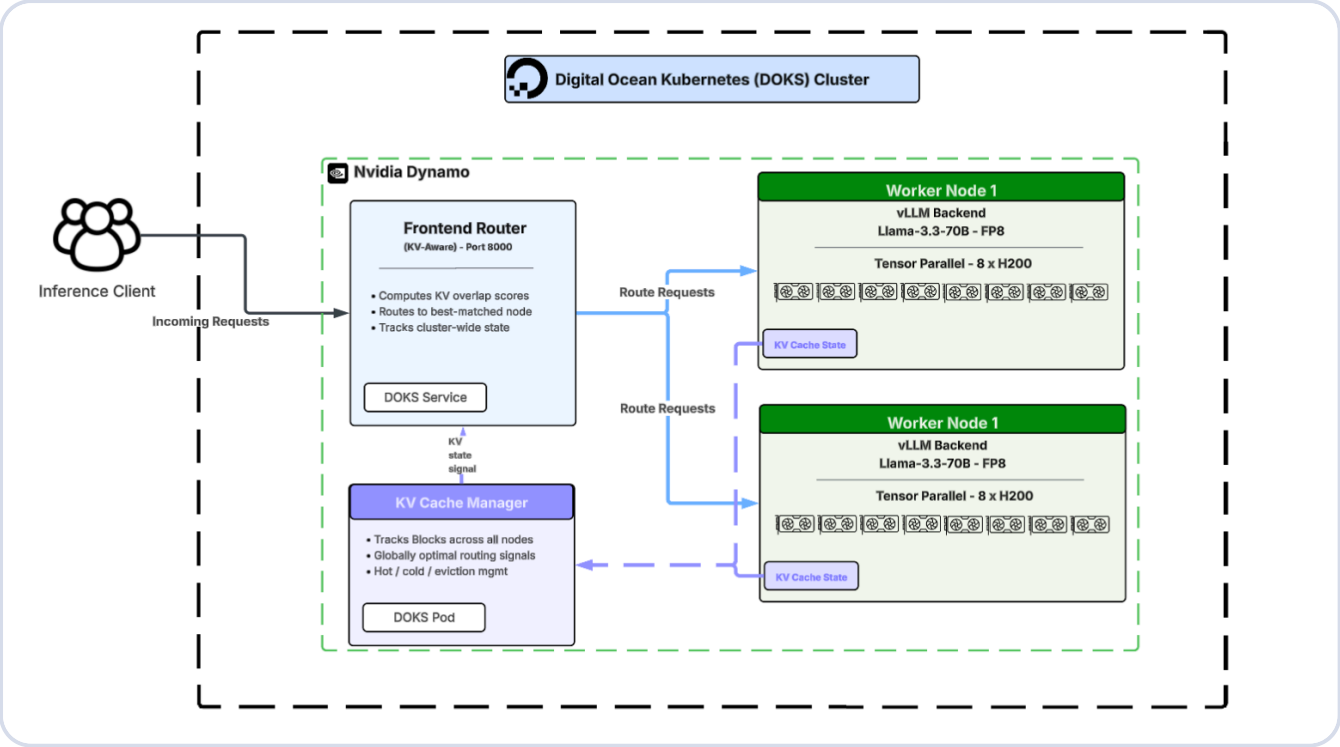
We ran the evaluation across a variety of DOKS clusters using **nvidia/Llama-3.3-70B-Instruct-FP8** as our model:

- 1-node with 8 NVIDIA H200 GPUs

- 2-node with 16 GPUs
- 4-node with 32 GUs

We also compared each topology for NVIDIA Dynamo + vLLM (chosen due to its broad feature coverage and clean integration surface) across two configurations: one with no KV-aware routing and one with KV-aware routing enabled. This would give Workato a real map of the performance-to-cost trade off space, not just a single data point.

The full cluster architecture for the 2-node deployment looked like this:



The frontend router of NVIDIA Dynamo is a lightweight DOKS service sitting in front of the GPU worker pods. The workers are full vLLM instances instrumented with Dynamo agents that continuously publish KV cache state back to the router and the cache manager. The KV cache manager is a separate pod that maintains the global view, tracking what's hot on which node, signaling evictions, and feeding the router's scoring function with fresh state. Separating the router, workers, and cache manager as distinct DOKS workloads allowed Workato to tune each layer independently and enabled the architecture to scale horizontally as workload demands grow.

The Two Configurations Tested

Configuration 1: No KV-Aware Routing

This configuration used the same model, same GPU, but with vLLM's own optimizations enabled: prefix caching, chunked prefill, FP8 weights and KV cache. This **does not**

include KV-aware cluster routing—each node makes smart local decisions, but there is no global view of cache state across the cluster.

Copy

```
vllm serve nvidia/Llama-3.3-70B-Instruct-FP8 \
  --tensor-parallel-size 8 \
  --enable-prefix-caching \
  --enable-chunked-prefill \
  --quantization fp8 \
  --kv-cache-dtype fp8 \
  --trust-remote-code
```

What actually happens when this configuration was put under a 100K-token prefill load? When two prompts share a long common prefix but land on different workers, both nodes redundantly recompute the same prefill because neither is aware of the work the other has already performed. Under this scenario, cache locality is entirely accidental, meaning that whether a request benefits from a warm KV cache is a matter of pure luck rather than strategic placement. This lack of coordination causes severe load imbalances to emerge; some workers become bogged down by long prefills while others manage decode-heavy traffic, causing their processing queues to diverge. Ultimately, tail latency explodes at higher concurrency levels because a single unlucky routing decision can pin a specific worker with multiple heavy prefills while other available resources sit idle.

Configuration 2: KV-Aware Routing

This is where the architecture fundamentally changes and out of the two configurations we tested, this was the clear winner. At a high level, the NVIDIA Dynamo deployment utilizes cluster-aware routing and splits the system into 1) Frontend (router + scheduler), 2) Set of workers (vLLM instances instrumented with Dynamo)

Dynamo Frontend (KV Router Enabled)

```
python3 -m dynamo.frontend --http-port 8000 --router-mode kv
```

The NVIDIA Dynamo frontend actively computes KV overlap scores against available workers and applies a specialized routing cost function to decide precisely where prefill and decode phases should execute. By tracking cluster-wide load and the current global cache state, the system ensures that every request is handled with maximum efficiency.

In this setup, the frontend is being run with KV routing fully enabled to validate these architectural improvements.

Workers (deployed as DOKS pods on GPU nodes):

Each worker is a vLLM instance launched via NVIDIA Dynamo's wrapper. The integration exposes -

- **KV cache events:** What blocks are created, reused, or evicted
- **Metrics:** Queue depth, decode load, memory pressure

Recipe: Dynamo + vLLM Worker

Copy

```
python3 -m dynamo.vllm \
  --model nvidia/Llama-3.3-70B-Instruct-FP8 \
  --tensor-parallel-size 8 \
  --enable-prefix-caching \
  --enable-chunked-prefill \
  --quantization fp8 \
  --kv-cache-dtype fp8 \
  --trust-remote-code \
  --enable-log-requests
```

The Core Idea: KV-Aware Routing

For each incoming request and each worker, NVIDIA Dynamo estimates a cost:

```
cost = overlap_score_weight * prefill_blocks + decode_blocks
```

Where:

- **prefill_blocks:** How much new KV must be computed (cache miss cost)
- **decode_blocks:** How busy the worker is on the decode side
- **overlap_score_weight:** Trades off TTFT vs decode latency

This turns routing from a **blind load balancer** into a **state-aware scheduler**.

Tuning

Up until this point, we've talked about why NVIDIA Dynamo + vLLM works: KV-aware routing, disaggregated prefill/decode, and cluster-level scheduling. But in practice, none of that matters unless the right knobs are turned.

We'll break this down using Workato's results and focus on three key metrics:

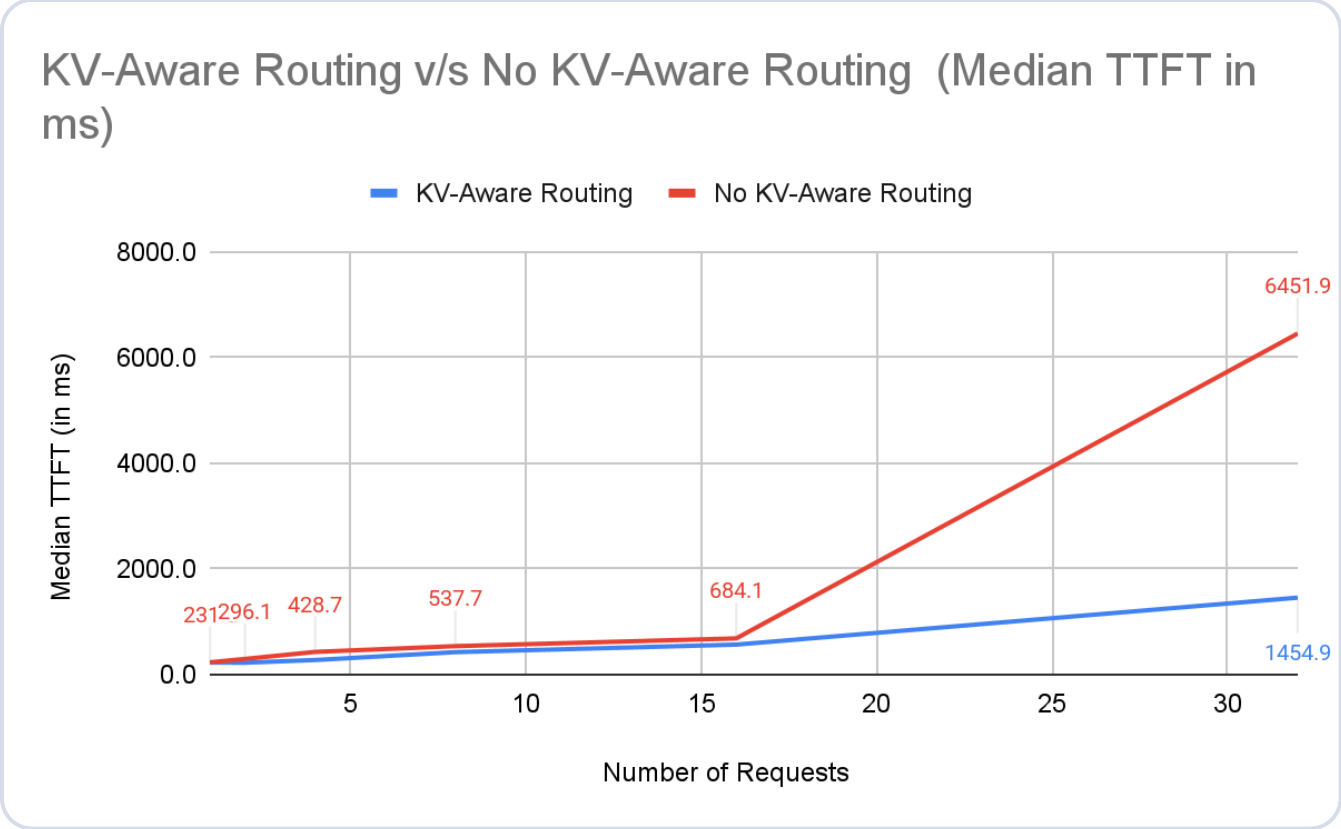
- **TTFT (Time-to-First-Token)**: How quickly the first token is produced after request arrival.
- **TPOT / ITL (Time-Per-Output-Token / Inter-Token Latency)**: How fast subsequent tokens are generated.
- **Throughput (QPS / Tokens/sec per GPU)**: Sustained work the system can handle.

TTFT: Prefill Cost & KV Reuse

Time-to-first-token (TTFT) is dominated by prefill work, the time the model spends processing the prompt to build KV states. Here, the tuning knobs are:

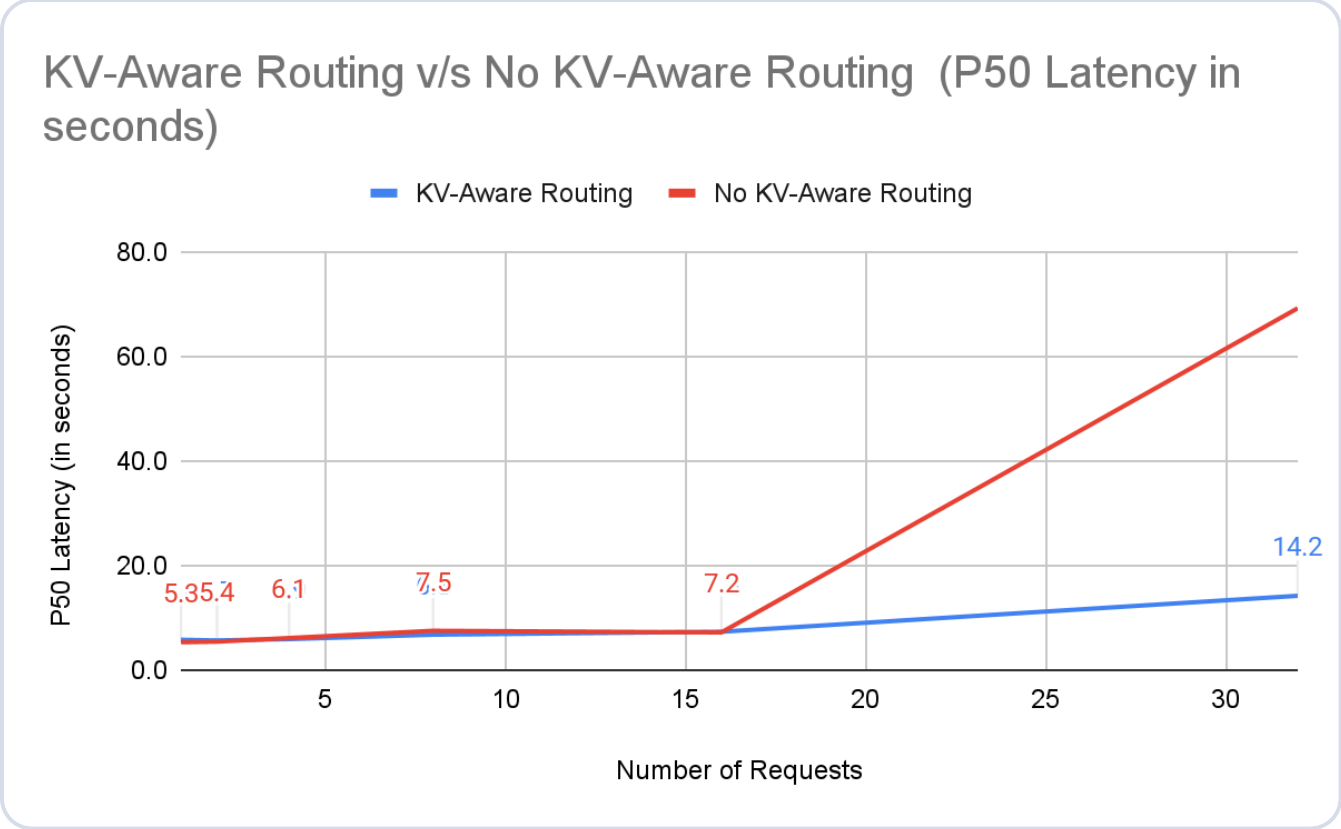
- `--enable-prefix-caching` : Reuses KV blocks for shared prefixes.
- `--enable-chunked-prefill` : Breaks prefill into manageable chunks to avoid GPU stalls.
- `--kv-cache-dtype fp8` : Reduces memory footprint and pressure, improving cache residency.

Why NVIDIA Dynamo excels: Dynamo routes requests to the worker with most of the KV prefix already cached, avoiding redundant prefill. Even under high concurrency (32 prompts), the TTFT advantage is clear.



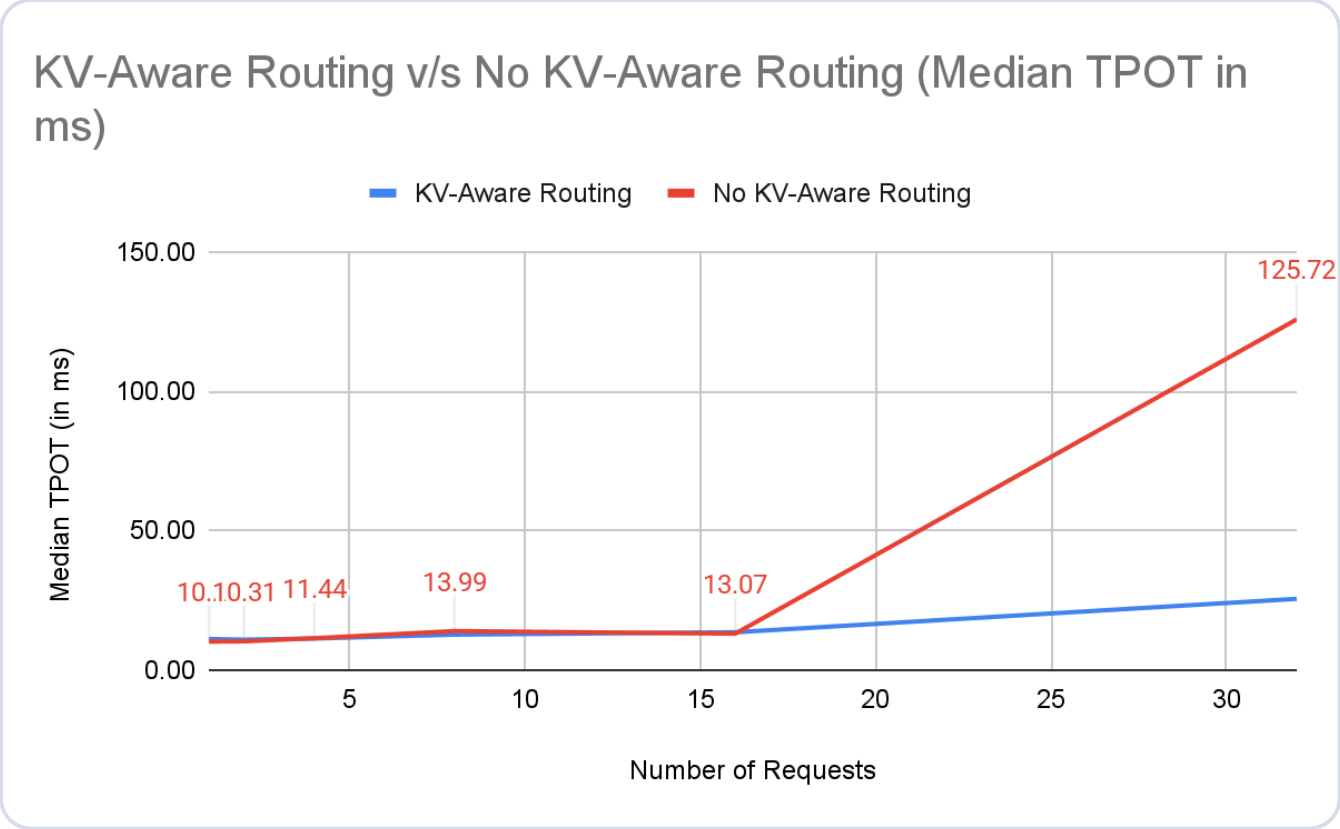
Median TTFT(ms) (2-node, 16–32 prompts):

Number of Requests	KV-Aware Routing with NVIDIA Dynamo	No KV-Aware Routing	% Improvement
16	566.7	684.1	+17.2%
32	1454.9	6451.9	+77.5%



P50 Latency(s) (2-node, 16–32 prompts):

Number of Requests	KV-Aware Routing with NVIDIA Dynamo	No KV-Aware Routing	% Improvement
16	7.3	7.2	-1.4%
32	14.2	69.2	+79.5%



TPOT / ITL: Decode Load Balancing

TPOT and ITL depend on how busy the GPU is during decoding. Chunked prefill prevents one heavy request from stalling the decode pipeline. NVIDIA Dynamo further balances the decode load across workers, keeping TPOT stable even under bursts.

Data snapshot (2-node, 16–32 prompts):

Number of Requests	KV-Aware Routing with NVIDIA Dynamo	No KV-Aware Routing	% Improvement
16	13.57	13.07	-3.8%
32	25.55	125.72	+79.97%

Without prefix-aware routing, naive vLLM collapses under high concurrency because some workers get "prefill-heavy" requests while others idle, inflating TPOT and tail latency.

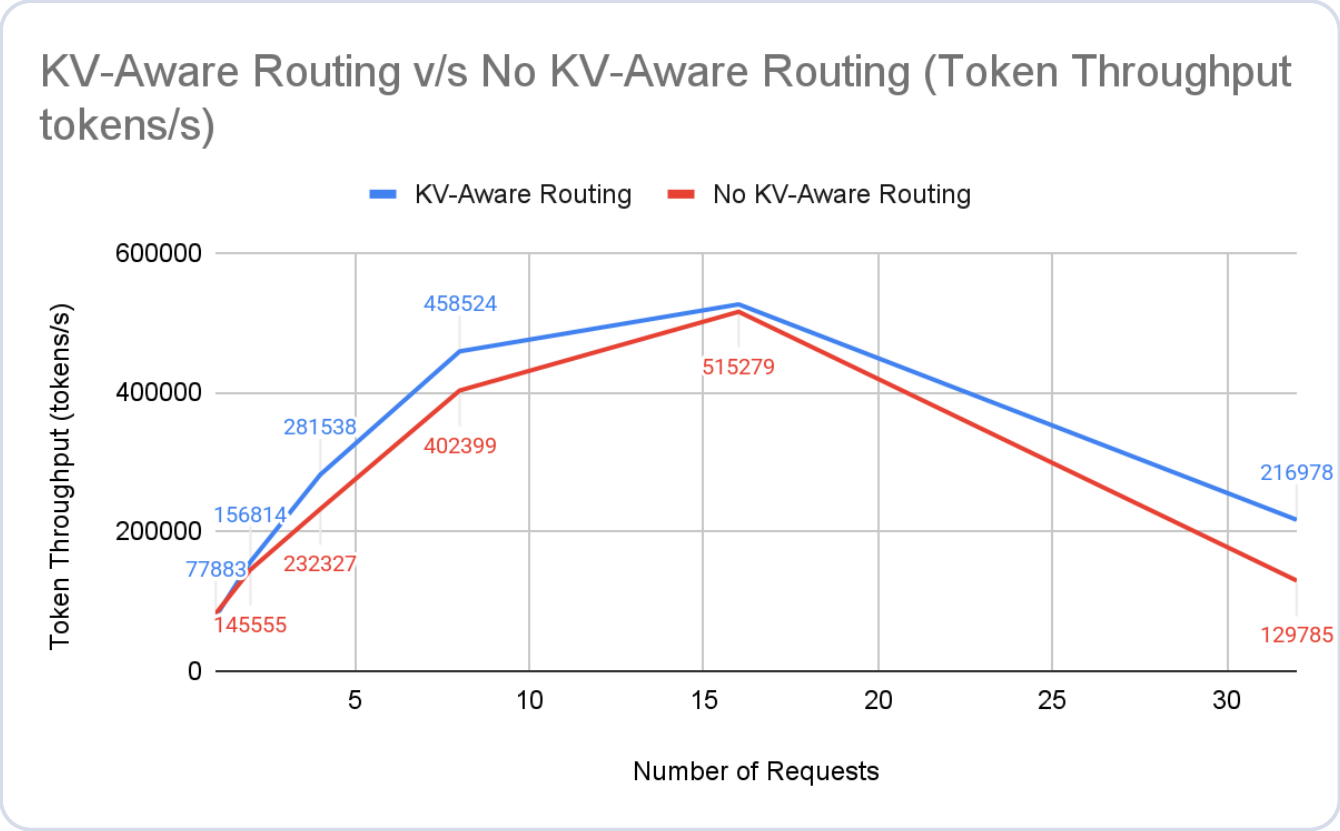
Where Dynamo wins:

- **KV reuse:** Avoids recomputing large prefill segments

- **Routing:** Sends requests to workers with hot KV caches
- **Scheduling**: Balances decode load, preventing hot GPU congestion

QPS & Token Throughput: GPU Utilization

Throughput per GPU at SLA (tokens/sec/GPU):



Data snapshot (2-node, 16–32 prompts):

Number of Requests	KV-Aware Routing with NVIDIA Dynamo	No KV-Aware Routing	% Improvement
16	525,898	515,279	+2.06%
32	216,978	129,785	+67.19%

With KV-aware routing, we achieved **13,561 tokens/sec per GPU versus 8,111 tokens/sec** without KV-aware routing, **a 67% uplift on identical hardware**. That improvement fundamentally changes capacity requirements. For a fixed SLA and token volume target, GPUs scale inversely with per-GPU throughput. A 1.67× throughput gain means each GPU now performs the work of 1.67 baseline GPUs, reducing required

capacity to roughly 60% of the original footprint or 40% fewer GPUs for the same workload. In practical terms, a deployment that previously required 10 GPUs could now sustain the same load with approximately 6.

Fewer GPUs means fewer nodes, lower hourly burn, and less scaling overhead. Higher throughput per GPU doesn't just make the system faster: it reduces the number of GPUs needed to serve the same load, converting performance gains into measurable dollar savings.

Conclusion

Workato's results on DigitalOcean's Agentic Inference Cloud show that inference performance at scale is determined less by the model itself and more by how the system is architected around it.

For long-context, high-concurrency workloads, redundant prefill computation and uneven decode load quickly become the dominant cost and latency drivers. Simply adding GPUs does not address these inefficiencies. Coordinated routing and cache-aware scheduling do.

By deploying on NVIDIA Dynamo with vLLM on DOKS and enabling KV-aware routing, Workato eliminated redundant prefill work, improved load balance across workers, and stabilized latency under concurrency. On identical hardware, this translated to 67% higher tokens/sec per GPU, up to 79% lower end-to-end latency under load, and 77% lower TTFT — resulting in lower cost per token and fewer GPUs required to meet SLAs.

The key takeaway is architectural: inference efficiency is primarily a systems problem. When routing, cache management, GPU topology, and Kubernetes orchestration are coordinated, gains compound.

About the author(s)

Rithish Ramesh Author
Senior Solutions Architect

See author profile

Karnik Modi Author
Sr. Manager, Solutions Architecture (AI)

See author profile

Karnik leads Solutions Architecture for AI and GPU initiatives at DigitalOcean, driving technical strategy and execution for strategic customers and partners operating large-

scale, production-grade workloads. He focuses on customer-first outcomes, cross-functional leadership, and delivering simple, powerful cloud platforms at scale.

Piyush Srivastava Author

See author profile

Principal Engineer, AI Infrastructure

Share

Engineering

Get in touch for GPUs and more Interested in improving your own inference costs with DigitalOcean's Agentic Inference cloud? Contact our team today.

Related Articles

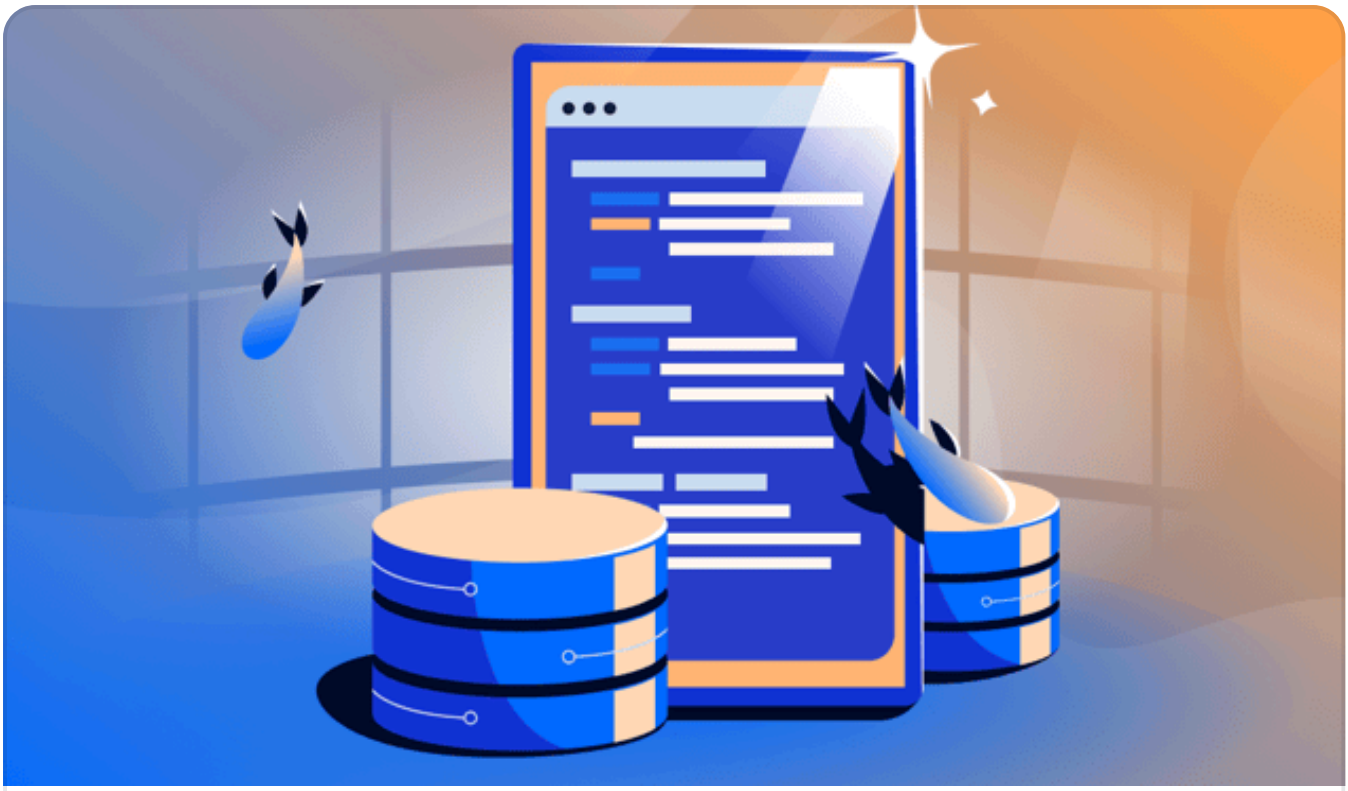
Engineering

DigitalOcean Gradient™ AI GPU Droplets Optimized for Inference: Increasing Throughput at Lower the Cost

Jason Peng

February 19, 2026 • 11 min read

Read more →



Engineering

LLM Inference Benchmarking - Measure What Matters

Piyush Srivastava

February 6, 2026 • 12 min read

Read more →

Engineering

Technical Deep Dive: How we Created a Security-hardened 1-Click Deploy OpenClaw

Freddie Rice

January 30, 2026 • 5 min read

Read more →

Company

Products

Resources

Solutions

Contact

© 2026 DigitalOcean, LLC. Sitemap. Cookie Preferences